

Operating System Learning Roadmap

A story-based, connected roadmap for fast revision and assessment preparation

Core Philosophy

Don't learn OS as isolated chapters. Follow one continuous story:

Real-world problem → Why it happens → OS mechanism → New problem → Next concept

The Master Story

1	Computer needs a manager	Why OS exists → Kernel → User/Kernel mode → System calls
2	You launch an application	Program → Process → PCB → Process states
3	Everyone wants the CPU	Schedulers → Context switching → CPU scheduling
4	One application has many workers	Threads → Multithreading
5	Workers share things and conflict	Race condition → Critical section → Mutex/Semaphore
6	Everyone waits for resources	Deadlock → Prevention/Avoidance/Detection
7	Processes need somewhere to live	Memory management → Paging → Segmentation
8	RAM is limited	Virtual memory → Demand paging → Page fault → Replacement
9	Programs need data and devices	File systems → Disk scheduling → I/O

Episode 0 — The Big Picture: Why Does an OS Exist?

Topics to learn

- Hardware resources: CPU, RAM, storage, I/O devices
- OS as resource manager
- OS as abstraction layer
- Kernel: privileged core of the OS
- User mode vs kernel mode
- System calls: controlled requests from programs to OS
- Interrupts and traps: how control reaches the OS

■ Real-world story	Real example: Your Java/FastAPI application wants memory, CPU time, file access and network access—but it should not directly control hardware.
■ Why this leads forward	This creates the central OS problem: many programs competing for limited resources.
■ Deep-dive checkpoint	Understand deeply: Kernel vs OS, user mode vs kernel mode, system call vs normal function call.
■ Memorization checkpoint	Memorize: main responsibilities of an OS; user mode vs kernel mode differences.
■ Practice checkpoint	No heavy numerical practice; use conceptual MCQs.

Episode 1 — You Launch an Application: Program Becomes Process

Topics to learn

- Program vs process
- What happens during process creation
- Process memory layout: code/text, data, heap, stack
- Process ID (PID)
- Process Control Block (PCB)
- Process states: new, ready, running, waiting/blocked, terminated
- State transitions

■ Real-world story	Real example: You run <code>`java MyApplication`</code> . The OS creates a process for the JVM, which loads and executes your Java code.
■ Why this leads forward	Multiple running instances require separate execution contexts → OS needs PCB and process states.
■ Deep-dive checkpoint	Understand deeply: Why one program can have multiple processes; PCB purpose; stack vs heap at a high level.
■ Memorization checkpoint	Memorize: standard process states and key PCB fields.
■ Practice checkpoint	Practice: process-state transition MCQs and simple scenarios.

Episode 2 — Everyone Wants the CPU: Scheduling

Topics to learn

- CPU burst and I/O burst
- Ready queue

- Long-term, short-term and medium-term schedulers
- Dispatcher
- Context switching
- Preemptive vs non-preemptive scheduling
- FCFS
- SJF
- SRTF
- Priority scheduling
- Round Robin
- Waiting time, turnaround time, response time
- Starvation and aging

■ Real-world story	Real example: Chrome, IntelliJ, Spotify and your Java server all need CPU time. On one core, the OS must choose who runs next.
■ Why this leads forward	Pausing a process requires saving its CPU context → PCB/context switching. Choosing the next process creates scheduling algorithms.
■ Deep-dive checkpoint	Understand deeply: Why each algorithm exists and its trade-offs; time quantum.
■ Memorization checkpoint	Memorize: formulas and algorithm characteristics.
■ Practice checkpoint	Practice heavily: Gantt charts, WT/TAT/RT, preemptive vs non-preemptive problems.

Episode 3 — One Application, Many Workers: Threads

Topics to learn

- Process vs thread
- Multithreading benefits
- Thread states
- User-level vs kernel-level threads (high level)
- Multithreading models
- Thread context switching
- Shared address space

■ Real-world story	Real example: A browser simultaneously handles UI, rendering, networking and background tasks.
■ Why this leads forward	Threads improve concurrency but shared memory introduces data consistency problems.
■ Deep-dive checkpoint	Understand deeply: Why threads are lighter than processes; what threads share vs keep private.
■ Memorization checkpoint	Memorize: process vs thread comparison.
■ Practice checkpoint	Practice: scenario-based MCQs.

Episode 4 — Workers Start Fighting: Synchronization

Topics to learn

- Concurrency vs parallelism
- Shared resource

- Race condition
- Critical section
- Critical section requirements: mutual exclusion, progress, bounded waiting
- Mutex
- Semaphore: binary and counting
- Spinlock (basic idea)
- Producer-consumer problem
- Readers-writers problem (basic)

■ Real-world story	Real example: Two concurrent requests update the same bank balance or inventory count.
■ Why this leads forward	Shared memory + concurrent execution → race condition → synchronization mechanisms.
■ Deep-dive checkpoint	Understand deeply: Mutex vs semaphore; why atomicity matters; why race conditions occur.
■ Memorization checkpoint	Memorize: critical-section requirements and core differences.
■ Practice checkpoint	Practice: trace simple synchronization scenarios; conceptual MCQs.

Episode 5 — Everyone Is Waiting: Deadlocks

Topics to learn

- Resource allocation
- Deadlock definition
- Coffman conditions
- Resource allocation graph
- Safe state vs unsafe state
- Deadlock prevention
- Deadlock avoidance
- Banker's algorithm
- Detection and recovery
- Starvation vs deadlock

■ Real-world story	Real example: Process A holds database lock 1 and waits for lock 2; Process B holds lock 2 and waits for lock 1.
■ Why this leads forward	Synchronization and resource ownership can create circular waiting → deadlock.
■ Deep-dive checkpoint	Understand deeply: Prevention vs avoidance; safe vs unsafe state.
■ Memorization checkpoint	Memorize: four Coffman conditions.
■ Practice checkpoint	Practice: Banker's algorithm and safe-sequence questions.

Episode 6 — Where Does a Process Live? Memory Management

Topics to learn

- Logical vs physical address
- Address binding
- Contiguous allocation

- Fixed vs variable partitions (overview)
- Internal fragmentation
- External fragmentation
- Compaction
- Paging
- Pages and frames
- Page table
- Address translation
- TLB
- Segmentation

■ Real-world story	Real example: Chrome, IntelliJ and JVM processes all need memory, but RAM is finite and fragmented.
■ Why this leads forward	Multiple processes need efficient memory allocation → contiguous allocation problems → paging and segmentation.
■ Deep-dive checkpoint	Understand deeply: Logical-to-physical translation; why paging solves external fragmentation.
■ Memorization checkpoint	Memorize: internal vs external fragmentation; paging vs segmentation.
■ Practice checkpoint	Practice: basic address translation and page-table questions.

Episode 7 — RAM Is Not Enough: Virtual Memory

Topics to learn

- Virtual memory concept
- Demand paging
- Page fault
- Page fault handling sequence
- Thrashing (concept)
- Locality of reference
- Page replacement
- FIFO
- Optimal
- LRU
- Belady's anomaly

■ Real-world story	Real example: Your system can run applications whose total memory demand exceeds currently available RAM.
■ Why this leads forward	Paging allows flexible placement; virtual memory extends apparent memory; missing pages cause faults; full RAM requires replacement.
■ Deep-dive checkpoint	Understand deeply: What actually happens during a page fault; why disk access is expensive.
■ Memorization checkpoint	Memorize: Belady's anomaly; characteristics of FIFO/LRU/Optimal.
■ Practice checkpoint	Practice heavily: page replacement fault-count problems.

Episode 8 — Saving Data and Talking to Devices: Files, Disk and I/O

Topics to learn

- File concept and metadata
- File operations
- Directory structures
- File allocation: contiguous, linked, indexed
- Disk scheduling: FCFS, SSTF, SCAN, C-SCAN
- I/O basics
- DMA (high level)
- Buffering, caching and spooling (basic differences)

■ Real-world story	Real example: Your application saves logs to disk while the OS coordinates storage and device access.
■ Why this leads forward	Processes generate persistent data and I/O requests → file system abstractions and device scheduling.
■ Deep-dive checkpoint	Understand deeply: Why different allocation/scheduling strategies have trade-offs.
■ Memorization checkpoint	Memorize: allocation method comparisons; disk scheduling direction concepts.
■ Practice checkpoint	Practice: a few disk-head movement problems and MCQs.

How to Study Each Episode

Step	What you do	Output
1. Recall	Before studying, write everything you remember about the topic.	Find memory gaps.
2. Story	Understand the real problem and why the OS needs a solution.	Conceptual intuition.
3. Connect	Ask: What new problem does this solution create?	Bridge to next topic.
4. Deep Dive	Pause only for concepts marked important.	Strong foundations.
5. Practice	Solve numericals/scenarios only where required.	Application skill.
6. Memorize	Add facts, formulas and comparisons to one quick sheet.	Fast recall.
7. Explain	Explain the concept without notes using your own example.	True understanding.

One-Week Suggested Tracking Plan

Day	Main Journey	Priority
Day 1	Episode 0 + Episode 1: Why OS exists → Program → Process → PCB → States	Foundation
Day 2	Episode 2: CPU scheduling + context switching + numericals	High
Day 3	Episode 3 + Episode 4: Threads → race conditions → synchronization	Very High
Day 4	Episode 5: Deadlocks + Banker's algorithm	High
Day 5	Episode 6: Memory management → paging → segmentation	Very High
Day 6	Episode 7 + Episode 8: Virtual memory → replacement → files/disk/I/O	High
Day 7	Full story recall + mixed MCQs + weak-topic repair	Revision

Final Success Checklist

- I can narrate the complete OS story without looking at notes.
- I understand why each major mechanism exists—not just its definition.
- I can explain Process → Thread → Synchronization → Deadlock as one connected chain.
- I can explain Memory → Paging → Virtual Memory → Page Fault → Replacement as one connected chain.
- I practiced CPU scheduling, Banker's algorithm and page replacement.
- I maintain one short 'Memorize This' sheet for formulas and easily confused comparisons.
- I can answer conceptual questions using real examples from Java/backend development.

Rule for our learning sessions: We will never move to a new concept merely because the syllabus says so. We move when the current problem naturally creates the need for the next concept. When depth, practice, or memorization is necessary, it will be explicitly marked.